

Synthetic Data Generation

- **Business Problem Statement:** Development and QA teams often lack realistic, privacy-compliant datasets for prototyping complex business applications. Manually defining relational schemas and populating them with dummy data is time-consuming and prone to logical inconsistencies.
- **Technical Solution:** Develop an AI-driven chatbot that automates the data generation lifecycle from requirements to generation:
 - **Schema Inference:** The system parses a raw business use case to automatically design a normalized database schema, identifying necessary **Tables** and their attributes.
 - **Relationship Mapping:** It intelligently defines **Primary Keys and Foreign Keys** to ensure referential integrity across the generated schema.
 - **Code Generation:** The output is an executable Python script utilizing the **faker library**, tailored to produce synthetic data that strictly adheres to the defined schema and business logic.

Text-to-SQL App

- **Business Problem Statement:** Non-technical stakeholders (e.g., product managers, marketing teams) struggle to derive direct insights from e-commerce databases due to a lack of SQL proficiency, creating a dependency on data analysts for ad-hoc queries.
- **Technical Solution:** Build a conversational interface powered by a Large Language Model (LLM) integrated with a **PostgreSQL** database containing **E-commerce data**:
 - **Query Translation:** The model accepts natural English queries and translates them into syntactically correct SQL.
 - **Execution Layer:** The system executes the generated SQL queries against the database to fetch real-time results.
 - **Natural Language Response:** The raw data results are processed and synthesized back into a human-readable English answer for the user.

Text-to-Visual Agent

- **Business Problem Statement:** Exploratory Data Analysis (EDA) requires significant manual effort to determine the right data aggregations and corresponding visualizations. Users need a way to instantly convert questions about data trends into accurate visual reports without writing boilerplate code.
- **Technical Solution:** Develop a specialized visualization agent using a **Local Language Model (preferably SLM)** focused on the [Spotify Global Music Dataset \(2009-2025\)](#):
 - **Standardized UI/UX:** Utilize a **Common Asset Library** to ensure consistent design patterns and visual styles across all outputs.
 - **Logic Engine:** The SLM processes English queries to determine three specific components:
 1. **Aggregation Logic:** The necessary Pandas operations (grouping, filtering, pivoting).
 2. **Visual Selection:** The appropriate chart type from the asset library.
 3. **Code Execution:** Generation and execution of the specific Python/Pandas code to render the final visual.

Underwriting Co-Pilot

- **Business Problem Statement:** Insurance underwriting is a high-stakes, manual process requiring strict adherence to complex guidelines. Human underwriters face fatigue and inconsistency, and testing automated solutions is difficult due to the scarcity of diverse, real-world claim scenarios that are safe to share for development.
- **Technical Solution:** Build a robust AI evaluation system anchored on the [NIOS Underwriting Manual](#) to automate and audit risk assessment:
 - **Synthetic Scenario Generation:** Leverage an LLM (e.g., ChatGPT) to ingest the specific underwriting PDF guidelines and generate a diverse dataset of synthetic applicants/cases that cover edge cases defined in the manual.
 - **Decision Engine (Prompt Engineering):** Develop a sophisticated prompting framework with strict **guardrails** that accepts a case file and issues an underwriting decision (Approve/Reject/Premium Load) based solely on the provided manual's rules.
 - **Automated Evaluation:** Implement an "**LLM as a Judge**" framework to score the Co-Pilot's decisions against the ground truth logic of the manual, ensuring accuracy and compliance.

Resume Parser and Job Matcher

- **Business Problem Statement:** The recruitment process is inefficient for both applicants and recruiters. Recruiters struggle to manually map skills from varying resume formats to Job Descriptions (JDs), while candidates often face rejection from Applicant Tracking Systems (ATS) without understanding gaps in their profile or knowing how to upskill.
- **Technical Solution:** Create a dual-facing recruitment intelligence tool that standardizes unstructured data and provides actionable feedback:
 - **Data Ingestion & Parsing:** Develop distinct parsers for **Resumes** and **Job Descriptions** to extract structured entities (skills, experience, education) from unstructured text.
 - **Matching Engine (ATS Scorer):** Implement an algorithm that compares the parsed resume entities against the parsed JD requirements to calculate a compatibility score (**ATS Score**).
 - **Feedback Loop:**
 - **Review:** Provide a qualitative analysis explaining *why* a resume received a specific score.
 - **Recommendation System:** Suggest specific **Projects** or certifications the candidate should undertake to bridge the identified gaps and improve their match rate.

Code Migration Assistant

- **Business Problem Statement:** Organizations are often locked into obsolete legacy technologies (e.g., COBOL mainframes, specific SQL dialects) that are expensive to maintain and incompatible with modern cloud infrastructures. Manual migration is high-risk, slow, and prone to human error, often resulting in "spaghetti code" that breaks critical business logic during translation.
- **Technical Solution:** Develop an intelligent transpilation pipeline that moves beyond simple text-based conversion to ensure functional equivalence:
 - **Structural Analysis (AST):** Instead of using regex or simple find-and-replace, parse the legacy source code to generate **Abstract Syntax Trees (AST)**. This captures the deep logical structure and intent of the code, independent of its syntax.
 - **Intelligent Transpilation:** Use the AST to map logic accurately to the target language (e.g., MySQL to PostgreSQL, COBOL to C), handling complex dialect differences and paradigm shifts (e.g., handling specific stored procedure logic or memory management).
 - **Validation:** Deploy an **"LLM as a Judge"** to evaluate the migration fidelity. The model compares the logic of the source and target code to verify that control flow and business rules are preserved without regression.

Background Verification & Diligence

- **Business Problem Statement:** Conducting thorough background checks for hiring, partnerships, or tenant vetting is tedious and prone to errors. Manual searches are often overwhelmed by noise—such as common names leading to mistaken identity—and can easily miss critical red flags buried in obscure news reports or public records, leading to poor risk management.
- **Technical Solution:** Build an autonomous investigative agent powered by the **Tavily Search API** (or similar tools) to aggregate, verify, and analyze public footprint data:
 - **Identity Resolution & Deduplication:** The system ingests multi-factor inputs (Social IDs, URLs, DOB, Father's Name) to perform strict entity resolution. It filters out "look-alike" profiles with the same name to ensure the data belongs to the correct individual.
 - **Risk & Sentiment Analysis:** The agent scans retrieved data for negative news coverage, legal notices, or concerning public behavior. It applies NLP to calculate a **Sentiment Score**, highlighting potential reputational risks.
 - **Comprehensive Reporting:** The final output is a holistic **User Summary** that includes a confidence-weighted **Match Score** (verifying it found the right person) and a risk assessment profile.

Fact & Claim Verification System

- **Business Problem Statement:** The rapid proliferation of AI-generated content and digital news has led to a surge in misinformation and LLM "hallucinations." Manually fact-checking every statement in dense documents or automated outputs is labor-intensive and unscalable, creating a critical need for automated tools to ensure content reliability and trust.
- **Technical Solution:** Develop an Agentic AI-driven fact-checking engine that validates text integrity against real-time data:
 - **Atomic Claim Extraction:** Decompose complex input text (articles, generated essays) into discrete, verifiable factual statements.
 - **Evidence Retrieval:** Integrate a web search tool to autonomously fetch real-world sources and cross-reference them against the extracted claims.
 - **Verification & Reporting:** Compare claims against the retrieved evidence to determine validity (True/False/Partially True) and generate a granular Accuracy Report complete with citations and confidence scores.

Campaign Asset Discovery Engine

- **Business Problem Statement:** Marketing teams possess vast archives of high-quality assets from previous campaigns that could be reused to save time and production costs. However, these assets are often unsearchable—stored with generic filenames or lacking metadata—making manual retrieval "close to impossible" and leading to unnecessary duplication of work.
- **Technical Solution:** Implement a privacy-first, semantic search system using Local Large Language Models (LLMs) to unlock the value of historical creative assets:
 - **Automated Tagging (VLM):** Utilize a local Vision-Language Model to process image archives and generate detailed, context-aware textual descriptions for every asset without sending data to the cloud.
 - **Vectorization:** Convert these generated descriptions into high-dimensional Embeddings to capture the semantic meaning of the visuals.
 - **Semantic Retrieval:** Build a search interface that allows teams to query the archive using natural language (e.g., "blue background with summer vibe"), instantly retrieving relevant past assets based on their content rather than filenames.